

Scenario 1

Student Marks Management System

A university examination department wants to develop a **Student Marks Management System** to store student marks electronically.

The examination staff should be able to enter the **student's name** and **marks**. Since the data is entered manually, the system must handle possible errors and invalid inputs.

The system should follow these rules:

1. The **student's name cannot be empty**. Use an assert statement to validate the name.
2. The **marks must be numeric**. If the user enters a character or string such as ABC, the program should handle the ValueError.
3. Marks must be between **0 and 100**.
4. Create a user-defined exception called InvalidMarksError.
5. If the marks are less than 0 or greater than 100, raise InvalidMarksError.
6. Valid student records should be stored in a file named student_records.txt.
7. The program should use try, except, and finally blocks to handle exceptions and ensure that the record-processing operation is completed.
8. The user should be allowed to enter details for **multiple students** using a loop.
9. After processing each student, ask whether the user wants to enter another record.

Problem Statement

Develop a Python-based Student Marks Management System for a university examination department. The system should accept a student's name and marks, validate the input using assertions and exception handling, raise a user-defined exception for marks outside the range 0–100, store valid records in a text file, and allow multiple student records to be processed until the user chooses to exit.

Scenario 2

A smart industrial monitoring system stores numerical sensor readings, such as temperature values, in a **NumPy array**: [10, 20, 30, 40, 50]

An operator needs to insert a new sensor reading at a specific position in the array. Since the system is designed to store **only numerical integer values**, the operator must not enter characters or text such as:

A
Hello
25.5

The system should validate the entered element using the isdigit() method.

Requirements

Develop a Python program that:

1. Creates a NumPy array containing numerical sensor readings.
2. Displays the original array.
3. Accepts the **insertion position** from the user.
4. Accepts a new element as input.
5. Uses isdigit() to check whether the entered element contains only digits.
6. Raises a ValueError if the user enters a character, string, decimal value, or any non-digit input.
7. Checks whether the insertion position is valid.
8. Raises an IndexError for an invalid position.
9. Inserts the valid numerical element into the NumPy array.
10. Uses try, except, and finally blocks for exception handling.

Problem Statement

A smart industrial monitoring system stores sensor readings in a NumPy array. Develop a Python program that allows an operator to insert a new sensor reading at a specified position. The program should ensure that only integer numerical values are inserted by validating the input using isdigit(). Handle invalid numerical input using ValueError and invalid insertion positions using IndexError. Display a completion message using the finally block.

Scenario 3

Precious Metal Loan Management

A financial institution provides loans against **Gold, Silver, and Platinum** ornaments. The interest rate is determined based on the **type of metal and its purity**.

The loan officer must provide the following details to calculate the interest:

1. **Metal type** – Gold, Silver, or Platinum
2. **Purity** –
 - Gold: 18K, 22K, or 24K
 - Silver: 925, 950, or 999
 - Platinum: 900, 950, or 999
3. **Loan amount**
4. **Loan tenure** in years

The bank wants to develop a Python program using a function:

`calculate_interest(metal, purity, amount, tenure)`

However, during loan processing, the loan officer may sometimes forget to provide one or more required details.

For example:

- The **metal type** may be missing.
- The **purity/carat** may be missing.
- The **loan amount** may be missing.
- The **loan tenure** may be missing.
- More than one argument may be missing.
- An invalid metal type or unsupported purity may be entered.

The program should use **exception handling** to identify and handle these situations without terminating abruptly.

Required Tasks

Write a Python program that:

1. Defines a function `calculate_interest(metal, purity, amount, tenure)`.
2. Determines the applicable interest rate based on the metal type and purity.
3. Calculates the interest using the provided loan amount and tenure.
4. Handles missing function arguments using **TypeError**.
5. Handles invalid metal types or unsupported purity values using **ValueError**.
6. Uses `try`, `except`, `else`, and `finally` blocks appropriately.

Sample Missing Argument Cases

Case	Function Call	
Missing metal type	<code>calculate_interest()</code>	
Missing purity	<code>calculate_interest("gold")</code>	
Missing amount	<code>calculate_interest("gold", 22)</code>	
Missing tenure	<code>calculate_interest("gold", 22, 500000)</code>	
Missing multiple arguments	<code>calculate_interest("silver")</code>	
Invalid metal	<code>calculate_interest("diamond", 999, 100000, 2)</code>	
Invalid purity	<code>calculate_interest("gold", 15, 100000, 2)</code>	